

APACHE SPARK SQL

Process structured and semi-structured data using SQL and DataFrame API



1. WHAT IS SPARK SQL?

Spark SQL is a module of Apache Spark used to work with structured and semi-structured data using SQL queries and the DataFrame API.



2. DOMAIN-SPECIFIC LANGUAGE (DSL)

A DSL is a language designed for a specific purposes.

DSL	Purpose
SQL	Querying and manipulating data
HTML	Creating web pages
CSS	Styling web pages

Spark SQL is a DSL for working with structured data.

3. COMPONENTS OF SPARK SQL



3.1 Hive Metastore Integration

- Connects to Apache Hive Metastore
- Access existing Hive tables
- Uses Hive metadata
- Provides interactive SQL shell



3.2 DataFrame API

- Distributed collection of data organized into rows and named columns
- Equivalent to a relational table in SQL



3.3 DataSource API

- Universal API for reading and writing structured data
- Supports multiple data sources like CSV, JSON, Parquet, ORC, Avro, JDBC, etc.

4. APPLICATIONS OF SPARK SQL



Banking

- Credit card fraud detection
- Customer analytics
- Risk analysis



Stock Market

- Real-time stock analysis
- Trading analytics



Healthcare

- Genomic sequencing
- Medical data processing



E-Commerce

- Customer behavior analysis
- Recommendation systems



Telecommunications

- Call detail record analysis
- Network monitoring

5. FEATURES OF SPARK SQL



Mix SQL queries with Spark programs. Supports Python, Scala, Java, and R.

```
df.createOrReplaceTempView("emp")
spark.sql("SELECT * FROM emp WHERE salary > 50000").show()
```



Works with Apache Hive. Read Hive tables, use metadata, and execute Hive SQL queries.



Uses Catalyst Optimizer for query optimization.

- Faster execution
- Efficient resource utilization
- Scalable for large datasets



Supports JDBC and ODBC connectivity. Connect external tools like Tableau, Power BI, Excel, SQL clients, etc.



Provides a common interface to access multiple data sources like Hive, JSON, CSV, Parquet, ORC, Avro, JDBC and can join data across them.

6. DATA SOURCES SUPPORTED



CSV



JSON



Parquet



ORC



Avro



Hive Tables

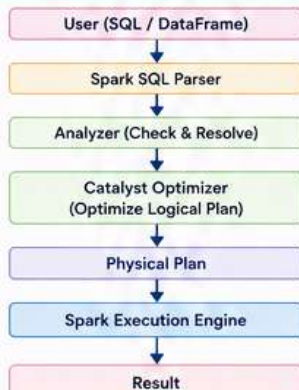


JDBC Databases



Text Files

7. HOW SPARK SQL WORKS?



8. CATALYST OPTIMIZER (WHY SPARK SQL IS FAST?)



Catalyst Optimizer

- ✓ Analyzes the query
- ✓ Removes unnecessary operations
- ✓ Reorders operations for efficiency
- ✓ Generates the best execution plan

Benefits



Faster Execution



Better Scalability



Efficient Resource Utilization



Lower Memory Usage

9. EXAMPLE

SQL Example

```
df.createOrReplaceTempView("employees")
spark.sql("""
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC
""").show()
```

DataFrame Example

```
df.filter(df["salary"] > 50000)
  .select("name", "salary")
  .orderBy(df["salary"].desc())
  .show()
```

10. SPARK SQL vs TRADITIONAL SQL

Feature	Traditional SQL	Spark SQL
Data	Works on data stored in a single machine	Works on distributed data across a cluster
Scalability	Limited scalability	Highly scalable
Performance	Slower on large datasets	Faster due to parallel processing & optimization
Data Sources	Usually works with relational databases	Works with multiple sources (CSV, JSON, Hive, Parquet, etc.)
Fault Tolerance	Limited	Built-in fault tolerance
Best For	Small to medium datasets	Big data and large-scale analytics

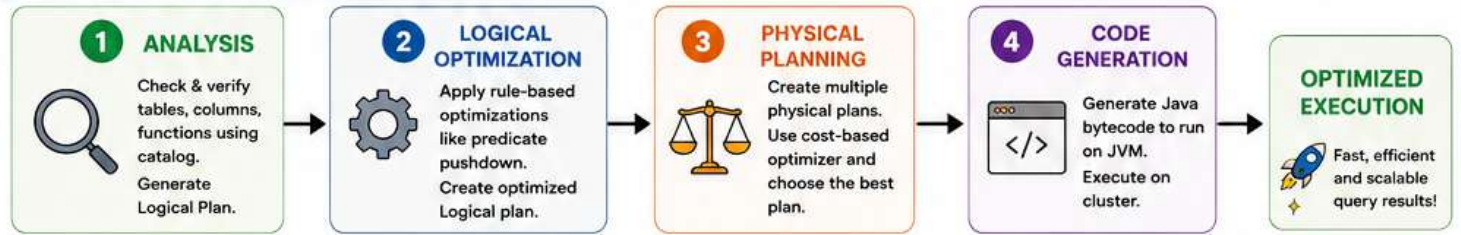
11. KEY TAKEAWAYS

- ★ Spark SQL provides a powerful DSL for structured and semi-structured data.
- ★ DataFrame API and DataSource API make data processing easy and efficient.
- ★ Catalyst Optimizer ensures high performance.
- ★ It supports multiple data sources and integrates easily with Spark programs.
- ★ Used in many industries for real-time and large-scale data analysis.



Catalyst Optimizer is the **optimization engine** of Spark SQL.
It optimizes SQL queries and DataFrame operations to make them run **faster and smarter**.

3 THE 4 STAGES OF CATALYST OPTIMIZER



4 STAGE BY STAGE EXPLANATION

1 ANALYSIS

- Generates Unresolved Logical Plan.
- Checks if tables, columns, functions exist.
- Uses Catalog (metastore) to resolve.
- Output: Logical Plan.

Real World Analogy
Teacher checks if student name exists in school records.

2 LOGICAL OPTIMIZATION

- Applies rule-based optimizations.
- Example: Predicate Pushdown.
- Filters data early at data source.
- Output: Optimized Logical Plan.

Real World Analogy
Shopkeeper keeps only required items in front, removes unnecessary ones.

3 PHYSICAL PLANNING

- Generates one or more physical plans.
- Uses Cost-Based Optimizer (CBO).
- Compares cost (CPU, memory, I/O).
- Selects plan with minimum cost.

Real World Analogy
Travel agent compares routes and chooses the cheapest & fastest one.

4 CODE GENERATION

- Converts best plan into Java bytecode.
- Bytecode runs on JVM.
- Executed on Spark Executors.
- Output: Final Result.

Real World Analogy
Chef gets final recipe and cooks the dish to serve customers.

5 MEMORY TRICK

Mnemonic: **A L P C**
Analyze → **L**ogically Optimize →
Plan Physically → **C**ode Generate

Remember in 10 Seconds

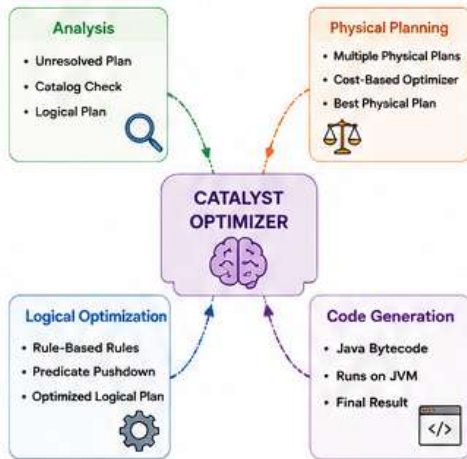


Story Method

Catalyst is like a smart assistant:
First it checks (A), then improves (L), then chooses best plan (P), and finally executes (C).



6 MIND MAP



7 ARCHITECTURE / SEQUENCE FLOW



8 EXPLAIN() EXAMPLE

```
df.filter("salary > 50000").explain(extended = true)
```

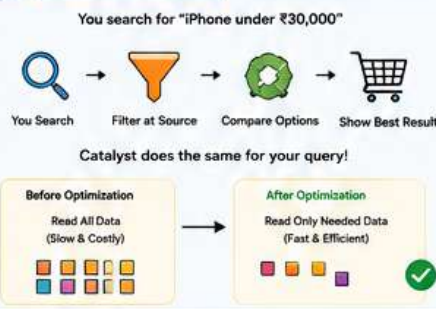
```
== Parsed Logical Plan ==
Filter (salary# > 50000)
+- Relation[emp]
```

```
== Analyzed Logical Plan ==
Filter (salary# > 50000)
+- Relation[emp]
```

```
== Optimized Logical Plan ==
Filter (salary# > 50000)
+- Relation[emp]
```

```
== Physical Plan ==
*(1) Filter (salary# > 50000)
+- *(1) Scan parquet emp
```

9 REAL-LIFE EXAMPLE



10 KEY FEATURES

Automatic Optimization

Faster Query Execution

Reduces Data Scanned

Better Resource Utilization

Works for SQL & DataFrames

Scalable on Big Data

11 COMMON MISTAKES

- Ignoring explain() output.
- Writing complex UDFs (hard to optimize).
- Not using filters early (no predicate pushdown).
- Selecting unnecessary columns.
- Not understanding data skew.

12 EXAM & INTERVIEW TIPS

- Know the 4 stages: Analysis, Logical Optimization, Physical Planning, Code Generation.
- Understand Predicate Pushdown with example.
- Explain Cost-Based Optimizer in Physical Planning.
- Use explain(extended = true) to show plans.
- Compare Rule-Based vs Cost-Based Optimization.



13. QUICK REVISION (IN 10 POINTS)

1 Catalyst is Spark SQL optimization engine.

2 Optimizes SQL & DataFrame code.

3 4 Stages: A, L, P, C

4 Analysis: Check & create logical plan.

5 Logical Optimization: Rule-based

6 Logical Optimization: Rule-based improvements.

7 Physical Planning: Cost-based best plan selection.

7 Code Generation: Java bytecode execution.

8 Use explain() to see all plans.

9 Result: Fast, efficient, scalable.

10 Remember: A L P C

REMEMBER: Catalyst makes your queries SMARTER, FASTER and BETTER!

1. IMPORT SPARK SESSION

```
from pyspark.sql import
SparkSession

spark = SparkSession.builder
    .appName("SparkSQL Demo")
    .getOrCreate()
```

SparkSession is the entry point for Spark SQL.

2. DATASET (CENSUS DATA)

age	workclass	fnlwgt	education	marital_status	income
39	State-gov	77516	Bachelors	Never-married	<=50K
50	Self-emp-not-inc	83311	Bachelors	Married-civ-spouse	>50K
38	Private	215646	HS-grad	Divorced	<=50K
53	Private	234721	11th	Married-civ-spouse	<=50K
28	Private	338409	Bachelors	Married-civ-spouse	>50K
...	...	...	...	...	...

Each row represents one person in the census dataset.

3. DEFINE SCHEMA

```
from pyspark.sql.types import *
schema = StructType([
    StructField("age", IntegerType(), True),
    StructField("workclass", StringType(), True),
    StructField("fnlwgt", LongType(), True),
    StructField("education", StringType(), True),
    StructField("education_num", IntegerType(), True),
    StructField("marital_status", StringType(), True),
    StructField("occupation", StringType(), True),
    StructField("relationship", StringType(), True),
    StructField("race", StringType(), True),
    StructField("sex", StringType(), True),
    StructField("capital_gain", IntegerType(), True),
    StructField("capital_loss", IntegerType(), True),
    StructField("hours_per_week", IntegerType(), True),
    StructField("native_country", StringType(), True),
    StructField("income", StringType(), True)
])
```

Manually defining schema gives better performance and accuracy.

4. READ DATA INTO DATAFRAME

```
df = spark.read.csv(
    "census.csv",
    schema=schema,
    header=True
)

df.printSchema()
```

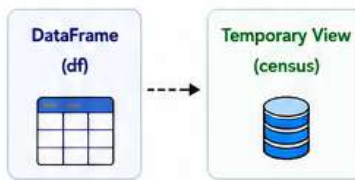
SCHEMA OF DATAFRAME

```
-- age: integer (nullable = true)
-- workclass: string (nullable = true)
-- fnlwgt: long (nullable = true)
-- education: string (nullable = true)
-- education_num: integer (nullable = true)
-- marital_status: string (nullable = true)
-- occupation: string (nullable = true)
-- relationship: string (nullable = true)
-- race: string (nullable = true)
-- sex: string (nullable = true)
-- capital_gain: integer (nullable = true)
-- capital_loss: integer (nullable = true)
-- hours_per_week: integer (nullable = true)
-- native_country: string (nullable = true)
-- income: string (nullable = true)
```

Data is now loaded as a Spark DataFrame.

5. REGISTER DATAFRAME AS TEMPORARY VIEW

```
df.createOrReplaceTempView("census")
```



Now we can run SQL queries on the DataFrame using the view name: census

6. RUN SIMPLE SQL QUERIES

6.1 View All Columns

```
spark.sql("SELECT *
FROM census").show(5)
```


6.2 Select Specific Columns

```
spark.sql("""
SELECT age, workclass,
income, income
""").show(5)
```

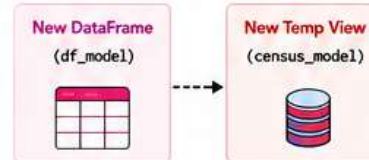

6.3 Create New Column

```
df_model = spark.sql("""
SELECT *, (fnlwgt * 1000)
AS new_column
FROM census
""")
```

Creates new DataFrame (df_model). Original DataFrame remains unchanged.

7. REGISTER NEW DATAFRAME AS VIEW

```
df_model.createOrReplaceTempView("census_model")
```



Use census_model for further SQL queries.

8. ADVANCED SQL QUERY (AGGREGATE FUNCTIONS)

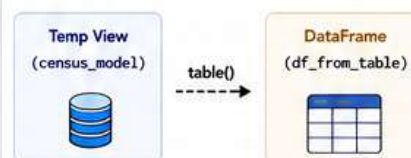
Calculate marital status rates by occupation using multiple functions.

```
SELECT
    occupation,
    ROUND(AVG(CASE WHEN marital_status = 'Married-civ-spouse' THEN 1 ELSE 0 END) * 100, 2) AS married_rate,
    ROUND(AVG(CASE WHEN marital_status = 'Widowed' THEN 1 ELSE 0 END) * 100, 2) AS widow_rate,
    ROUND(AVG(CASE WHEN marital_status = 'Divorced' THEN 1 ELSE 0 END) * 100, 2) AS divorce_rate,
    ROUND(AVG(CASE WHEN marital_status = 'Separated' THEN 1 ELSE 0 END) * 100, 2) AS separated_rate,
    ROUND(AVG(CASE WHEN marital_status = 'Never-married' THEN 1 ELSE 0 END) * 100, 2) AS bachelor_rate
FROM census_model
GROUP BY occupation
ORDER BY occupation;
```

SUM() AVG() ROUND() LTRIM() GROUP BY ORDER BY

9. TEMP VIEW TO DATAFRAME

```
df_from_table = spark.table("census_model")
df_from_table.printSchema()
```



We can convert any temp view back to DataFrame using table().

10. SQL QUERY FOR VISUALIZATION

Percentage of people with income > 50K within each occupation.

```
SELECT
    occupation,
    ROUND(AVG(CASE WHEN income = '>50K' THEN 1 ELSE 0 END)
    * 100, 2) AS high_income_percent

FROM census_model
GROUP BY occupation
ORDER BY high_income_percent DESC;
```



11. CONVERT TO PANDAS DATAFRAME

```
spark_df = spark.sql(query_above)
pandas_df = spark_df.toPandas()
pandas_df.columns = ['occupation', 'high_income_percent']
```

occupation	high_income_percent
Exec-managerial	48.21
Prof-specialty	46.35
Sales	44.97
Craft-repair	30.22
Transport-moving	28.11
...	...

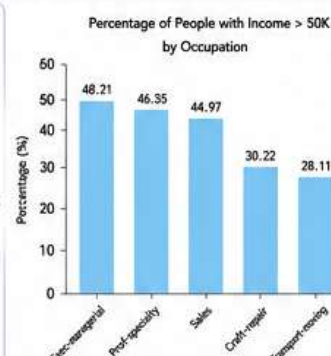


Convert only aggregated/small result for better performance.

12. VISUALIZATION (MATPLOTLIB)

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.bar(pandas_df['occupation'],
    pandas_df['high_income_percent'],
    color='skyblue')
plt.xticks(rotation=45, ha='right')
plt.ylabel('Percentage (%)')
plt.title('Percentage of People with Income > 50K
by Occupation')
plt.tight_layout()
plt.show()
```



13. KEY FEATURES

SQL on Big Data using Spark Easy integration with DataFrames Supports complex SQL functions High performance & scalable Seamless Python integration Works with structured data



A → **L** → **P** → **C**

Analysis → Logical Optimization
→ Physical Planning → Code Generation
(The 4 stages of Catalyst Optimizer)

CatCatalyst Optimizer analyzes your query and makes it FASTER & SMARTER!

